

scala and spark cheatsheet

◆ 1. What is Scala?

👉 **Scala = Scalable Language**

- Runs on **Java Virtual Machine (JVM)**
- Combines:
 - **Object-Oriented Programming (OOP)**
 - **Functional Programming (FP)**

💡 Think:

Java + Python + Functional Programming → Scala

◆ 2. VARIABLES (val vs var)

🧠 **Core Idea**

Keyword	Mutable?	Use Case
val	✗ No	Default choice (safe)
var	✓ Yes	When value must change

✓ Examples

```
1  val x = 10           // immutable
2  var y = 20           // mutable
3
4  y = 30               // allowed
5  // x = 15 ✗ error
```

◆ Type Inference (VERY IMPORTANT)

```
1  val a = 100          // Int automatically
2  val b = "Scala"      // String
3  val c = 3.14          // Double
```

You can also write:

```
1 val d: Int = 50
```

Best Practice

 Use `val` 90% of the time

Why?

- Prevent bugs
- Makes code predictable (functional style)

3. BASIC DATA TYPES

```
1 val i: Int = 10
2 val d: Double = 2.5
3 val s: String = "Hello"
4 val b: Boolean = true
5 val c: Char = 'A'
```

4. SIMPLE COLLECTIONS

Scala collections are **powerful + functional**.

4.1 List (Immutable)

```
1 val nums = List(1, 2, 3, 4)
```

Common Operations

```
1 nums.head           // 1
2 nums.tail           // List(2,3,4)
3 nums.length         // 4
4
5 nums.map(x => x * 2)
6 // List(2,4,6,8)
7
8 nums.filter(x => x % 2 == 0)
9 // List(2,4)
```

4.2 Array (Mutable)

```
1 val arr = Array(1, 2, 3)
2 arr(0) = 10
```

◆ 4.3 Set (Unique values)

```
1 val s = Set(1, 2, 2, 3)
2 // Set(1,2,3)
```

◆ 4.4 Map (Key-Value)

```
1 val m = Map(
2   "a" -> 1,
3   "b" -> 2
4 )
5
6 m("a") // 1
```

🔥 Most Important Concept (Interview + Real Use)

👉 Collections + functions:

```
1 val result = nums
2   .filter(_ % 2 == 0)
3   .map(_ * 10)
```

💡 This is **functional chaining**

◆ 5. CONDITIONALS

◆ 5.1 if-else (Expression!)

```
1 val x = 10
2
3 val result = if (x > 5) "Big" else "Small"
```

💡 In Scala:

`if` returns a value (like Python)

◆ 5.2 Nested

```
1 val grade =  
2   if (x > 90) "A"  
3   else if (x > 75) "B"  
4   else "C"
```

🔥 Pattern Matching (VERY IMPORTANT)

Scala's superpower:

```
1 val num = 2  
2  
3 num match {  
4   case 1 => "One"  
5   case 2 => "Two"  
6   case _ => "Other"  
7 }
```

💡 Better than switch-case in Java

◆ 6. LOOPS

◆ 6.1 for loop

```
1 for (i <- 1 to 5) {  
2   println(i)  
3 }
```

◆ 6.2 for with collection

```
1 val nums = List(1,2,3)  
2  
3 for (n <- nums) {  
4   println(n)  
5 }
```

◆ 6.3 Yield (VERY POWERFUL)

```
1 val squares = for (n <- nums) yield n * n  
2 // List(1,4,9)
```

◆ 6.4 while loop

```
1  var i = 0
2
3  while (i < 5) {
4      println(i)
5      i += 1
6  }
```

⚠ Avoid when possible → prefer functional style

◆ 7. FUNCTIONS

◆ 7.1 Basic Function

```
1  def add(a: Int, b: Int): Int = {
2      a + b
3  }
```

◆ 7.2 Short Syntax

```
1  def add(a: Int, b: Int) = a + b
```

◆ 7.3 Anonymous Functions (VERY IMPORTANT)

```
1  val multiply = (x: Int) => x * 2
```

◆ 7.4 Functions with Collections

```
1  val nums = List(1,2,3)
2
3  nums.map(x => x * 2)
4  nums.filter(x => x > 1)
```

🔥 7.5 Higher-Order Functions

Function taking another function:

```
1 def applyFunc(x: Int, f: Int => Int) = f(x)
2
3 applyFunc(5, x => x * 2) // 10
```

◆ 8. REAL-WORLD STYLE CODE

```
1 val nums = List(1,2,3,4,5)
2
3 val result = nums
4   .filter(_ % 2 == 1)
5   .map(_ * 10)
6   .sum
7
8 println(result)
```

◆ 9. KEY SCALA MINDSET (VERY IMPORTANT)

If you want to become strong:

✗ Avoid:

- Too many `var`
- Imperative loops

✓ Prefer:

- `val`
- `map`, `filter`, `reduce`
- Immutability

◆ 10. QUICK CHEAT SHEET

```
1 // variable
2 val x = 10
3 var y = 20
4
5 // list
6 List(1,2,3).map(_*2)
7
8 // condition
9 if (x > 5) "Yes" else "No"
10
11 // loop
12 for (i <- 1 to 5) println(i)
```

```
13
14 // function
15 def f(x: Int) = x * 2
```

MASTER SPARK TEMPLATE (USE THIS EVERYWHERE)

Built on  Apache Spark

CONSTANT CODE (ALMOST NEVER CHANGES)

```
1 import org.apache.spark.sql.SparkSession
2
3 object StreamingApp {
4   def main(args: Array[String]): Unit = {
5
6     val spark = SparkSession.builder
7       .appName("StreamingApp")
8       .master("local[*]")
9       .getOrCreate()
10
11     import spark.implicits._
12
13     val lines = spark.readStream
14       .format("socket")
15       .option("host", "localhost")
16       .option("port", 9999)
17       .load()
18
19     val data = lines.as[String]
20
21     // ● CHANGE ONLY BELOW
22     val result = ???
23
24     val query = result.writeStream
25       .outputMode("append") // default safe
26       .format("console")
27       .option("truncate", false)
28       .start()
29
30     query.awaitTermination()
31   }
32 }
```

TOP 10 LAB QUESTIONS (VERY LIKELY)

1. Print Input (No Change)

```
1 val result = data
```

👉 Just prints whatever you type

2. Convert to Uppercase

```
1 val result = data.map(_.toUpperCase)
```

3. Convert to Lowercase

```
1 val result = data.map(_.toLowerCase)
```

4. Filter Lines Containing "ERROR"

```
1 val result = data.filter(_.contains("ERROR"))
```

5. Filter Even Numbers

```
1 val result = data
2   .map(_.toInt)
3   .filter(_ % 2 == 0)
```

👉 Input:

```
1 1
2 2
3 3
4 4
```

6. Filter Odd Numbers

```
1 val result = data
2   .map(_.toInt)
3   .filter(_ % 2 != 0)
```

7. Numbers Greater Than 50

```
1  val result = data
2    .map(_ .toInt)
3    .filter(_ > 50)
```

8. Multiply Numbers by 10

```
1  val result = data
2    .map(_ .toInt)
3    .map(_ * 10)
```

9. Word Count (IMPORTANT)

```
1  val result = data
2    .flatMap(_ .split(" "))
3    .groupBy("value")
4    .count()
```

 CHANGE output mode:

```
1  .outputMode("complete")
```

10. Count Total Lines

```
1  val result = data.groupBy().count()
```

 use:

```
1  .outputMode("complete")
```

OUTPUT MODE RULE (REMEMBER)

Case	Mode
map/filter	append 
groupBy/count	complete 



COMMON SAFE VERSION (NO CRASH)

If unsure input type:

```
1 import scala.util.Try
2
3 val result = data
4   .map(x => Try(x.toInt).toOption)
5   .filter(_ != null)
6   .map(_.get)
7   .filter(_ % 2 == 0)
```



HOW TO RUN (UBUNTU FULL FLOW)

◆ STEP 1: Start Netcat

```
1 nc -lk 9999
```

👉 Keep this running

◆ STEP 2: Save Scala File

```
1 nano StreamingApp.scala
```

Paste full template + scenario

◆ STEP 3: Run Using spark-shell (EASIEST)

```
1 spark-shell -i StreamingApp.scala
```

◆ OR Run Using spark-submit

```
1 scalac StreamingApp.scala
2 spark-submit --class StreamingApp StreamingApp.jar
```

(only if needed)



HOW EXECUTION WORKS

```
1 Terminal 1 → nc -lk 9999
2 Terminal 2 → spark-shell runs code
3 Type in Terminal 1 → Output in Terminal 2
```



SAMPLE TEST

Type in nc:

```
1 10
2 20
3 30
```



Spark prints result live



COMMON LAB ERRORS (IMPORTANT)

- ✗ Forgot to start nc
- ✗ Wrong port
- ✗ Used complete for map
- ✗ Input not numeric for `.toInt`



WHAT TO SAY IN LAB VIVA

"We use socket streaming via netcat. Spark reads it as a stream and processes data using transformations like map and filter, then outputs results continuously."



5-SECOND REVISION

```
1 readStream → as[String] → logic → writeStream
```